

Database Optimization Techniques

 systemdesignschool.io/fundamentals/database-optimization



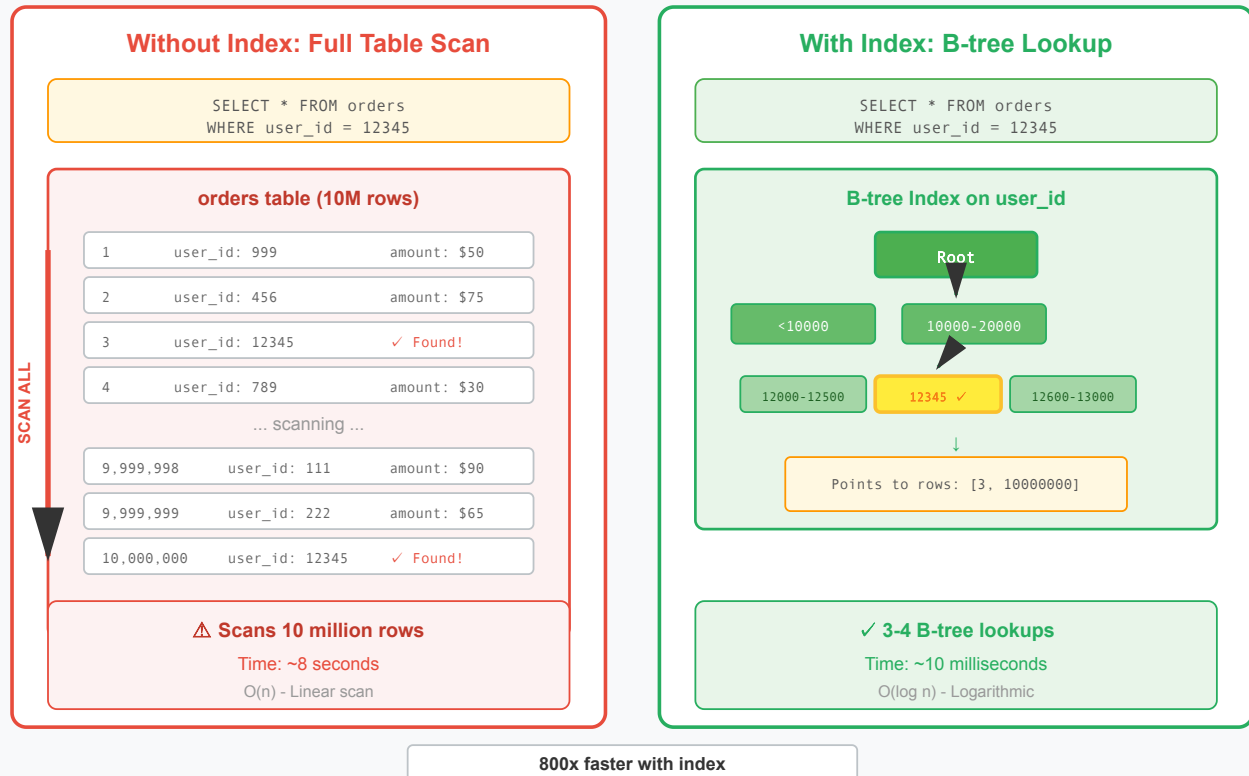
A database query scans all 10 million rows to find orders for user 12345. Response time is 8 seconds. Users abandon the page. An optimized database uses indexes, sharding, replication, and tiered storage to answer the same query in 10 milliseconds.

After you design the data model, you optimize how the database stores and retrieves that data. An unoptimized database forces full table scans for every query. An optimized database handles millions of queries per second.

Indexing

An index creates a fast lookup structure so queries avoid scanning the entire table. Without an index on `user_id`, the query `SELECT * FROM orders WHERE user_id = 12345` scans every row. With an index, the database jumps directly to rows for that user using a B-tree structure.

Database Indexing: Full Table Scan vs Index Lookup



A single-column index speeds up queries that filter on one field. If you frequently query by `user_id`, create an index on that column. A multi-column index (also called a composite index) speeds up queries that filter on multiple fields together. If you frequently query by `user_id` and `status`, create an index on `(user_id, status)`. The order matters: an index on `(user_id, status)` helps queries filtering on `user_id` alone or on both fields, but not queries filtering only on `status`.

Indexes work best when they reduce the candidate set significantly. An index on `user_id` in a table with 1 million users works well because each user has few rows. An index on `status` with values `active`, `inactive`, `pending` works poorly because each value matches 33% of rows. The database still scans hundreds of thousands of rows.

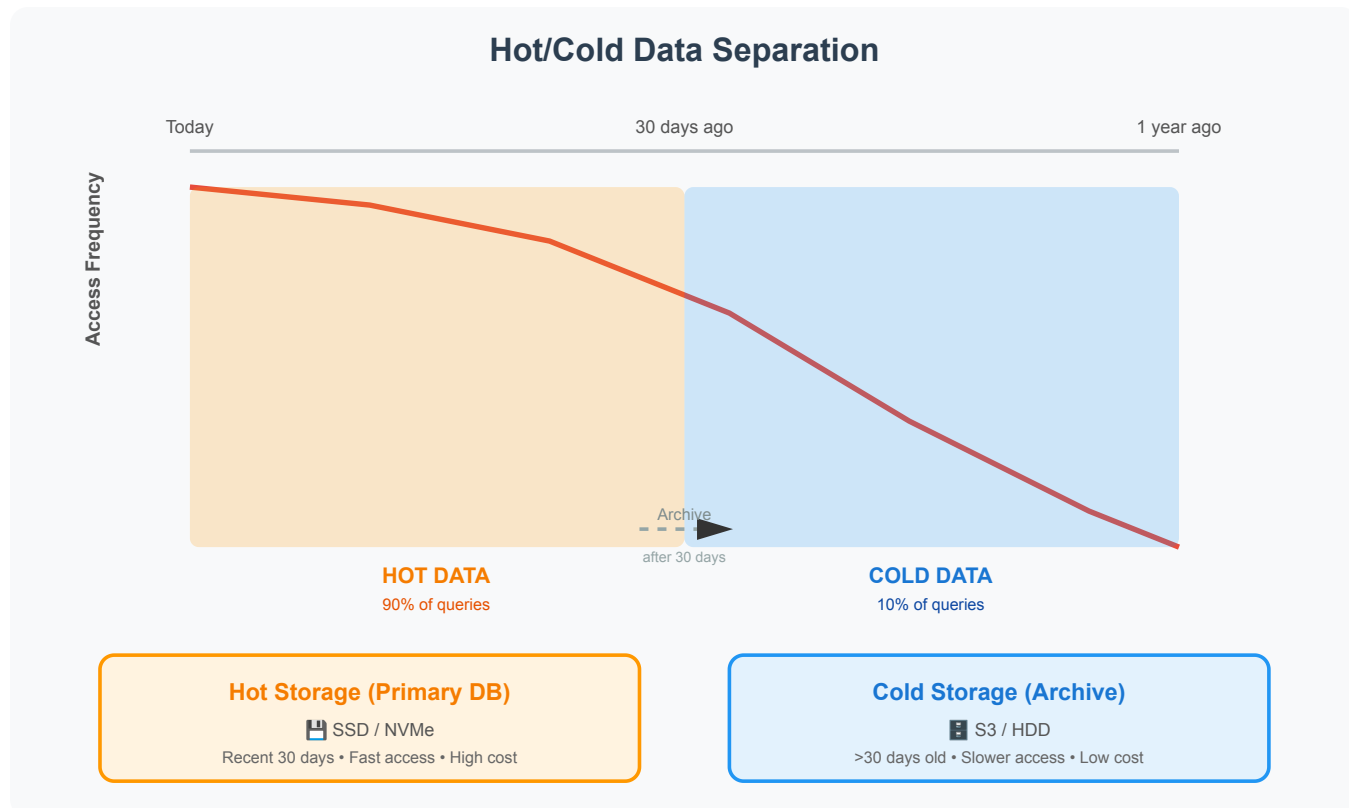
Indexes speed up reads but slow down writes. Every insert, update, or delete must update all relevant indexes. Only index fields critical to query performance. Use `EXPLAIN` to identify slow queries before adding indexes. In interviews, mention that over-indexing wastes memory and degrades write performance.

Partitioning and Replication

[Database partitioning](#) and [replication](#) are covered extensively earlier in the course. Partitioning splits data across multiple servers to handle more writes and storage. Replication copies data to multiple servers for availability and read scaling. These techniques are fundamental to scaling databases, but using them effectively requires additional optimizations covered here.

Cold and Hot Data Separation

Hot data is accessed frequently. Cold data is accessed rarely. Storing both on the same fast, expensive storage wastes money. Separate them by access patterns.



Recent orders are hot data. Users check order status frequently in the first few days. Store recent orders on fast SSDs in the primary database. Orders older than 30 days are cold data. Users rarely view them. Archive cold orders to cheaper storage like HDDs or Amazon S3. The application checks the primary database for recent orders first. If not found, it queries the archive.

This tiering reduces database size and improves performance. A smaller database fits more data in memory for caching. Cold data queries might be slower, but that is acceptable because they are rare.

Query Optimization

Beyond indexes, query optimization shapes how the database executes operations. Use **EXPLAIN** to understand query execution plans. A query that scans 1 million rows when it should scan 1,000 indicates missing indexes or poor query structure.

Query Optimization Strategies

Without Optimization: Expensive JOIN

customers			orders	
id	name		id	cust_id
1	Alice	JOIN	101	1
2	Bob		102	2
3	Charlie		103	1

```
SELECT orders.*, customers.name
FROM orders JOIN customers
ON orders.cust_id = customers.id
```

⚠ Slow: Scans both tables every time

Denormalization: Store Redundant Data

orders (denormalized)		
id	cust_id	cust_name
101	1	Alice
102	2	Bob
103	1	Alice

```
SELECT * FROM orders
WHERE id = 101
```

✓ Fast: Single table lookup

Materialized Views: Pre-compute Expensive Aggregations

sales (millions of rows)			
id	region	amount	date
1	US	100	2024-01-01
2	EU	200	2024-01-01
3	US	150	2024-01-02
... (millions more)			

⚠ Slow aggregation query

Pre-compute
hourly

monthly_sales_by_region		
month	region	total
2024-01	US	45000
2024-01	EU	38000
2024-02	US	52000
2024-02	EU	41000

✓ Fast: Pre-computed

Dashboard Query

```
SELECT * FROM
monthly_sales_by_region
WHERE month = '2024-01'
```

10ms instead of 30s

3000x faster ✓



Denormalization trades storage for query performance. Instead of joining orders and customer tables on every query, store customer name directly in the orders table. Reads become faster. Writes become more complex because customer name updates must propagate to all orders. Use denormalization when read frequency far exceeds write frequency.

Materialized views pre-compute expensive queries. A dashboard showing monthly sales by region recalculates aggregations on every load. A materialized view computes this once and stores results. Queries read the pre-computed view instead of scanning millions of rows. Refresh the view hourly or when source data changes.

Interview Application

When asked to design a database for a social network, mention indexing user lookups by **user_id** and posts by **created_at**. Reference partitioning and replication from earlier in the course for horizontal scaling. Focus on optimizations specific to the use case: denormalize user data into posts for faster timeline rendering, use materialized views for trending topics, separate hot and cold data by archiving posts older than one year.

Explain trade-offs clearly. Indexes speed reads but slow writes. Denormalization improves read performance but complicates updates. Materialized views trade storage and refresh cost for faster queries. Cold storage reduces costs but increases latency for old data. Show you understand when each optimization applies and what it costs.